Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY 2026–:
Name & class	Varshitha Darisa AIDS-'A'	27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_{i} y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display model.summary().

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

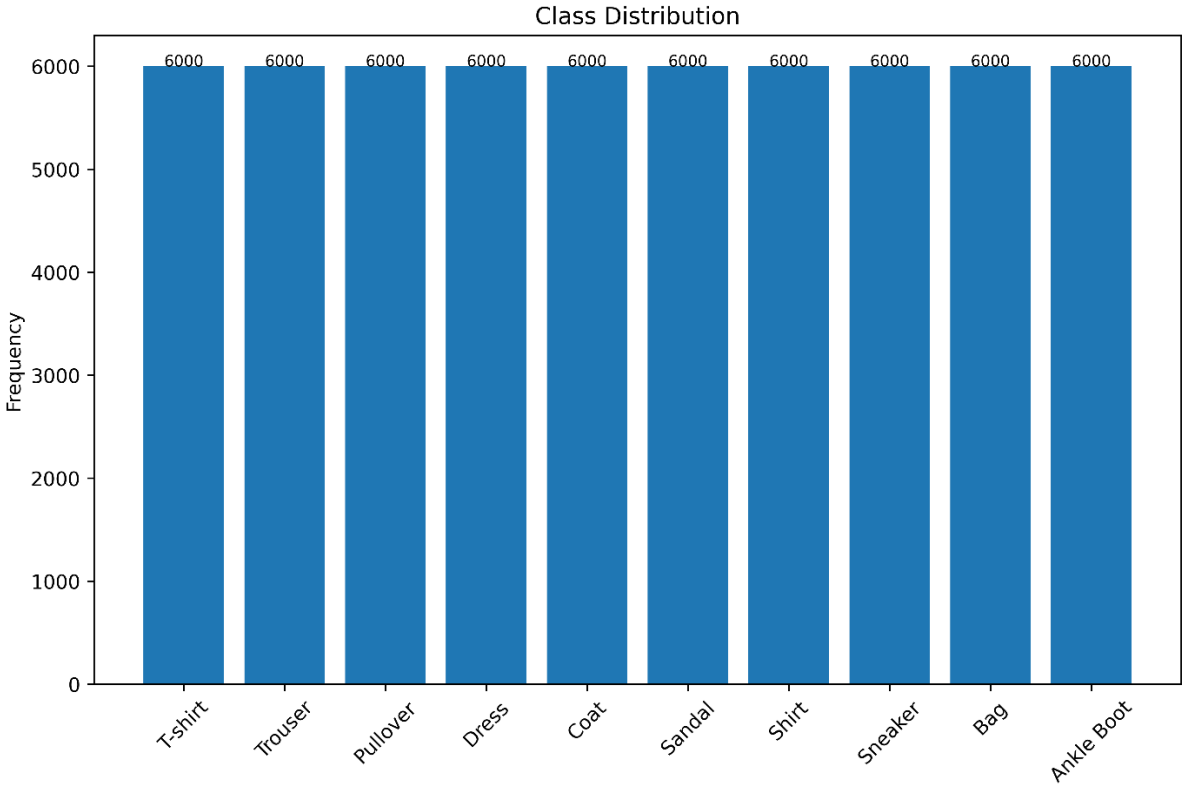
1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Mandatory Plots

1. Sample Images

Inference: The sample images show different categories of clothing items present in the Fashion-MNIST dataset. The images are grayscale and have different patterns, which helps the model learn the features of each class during training.

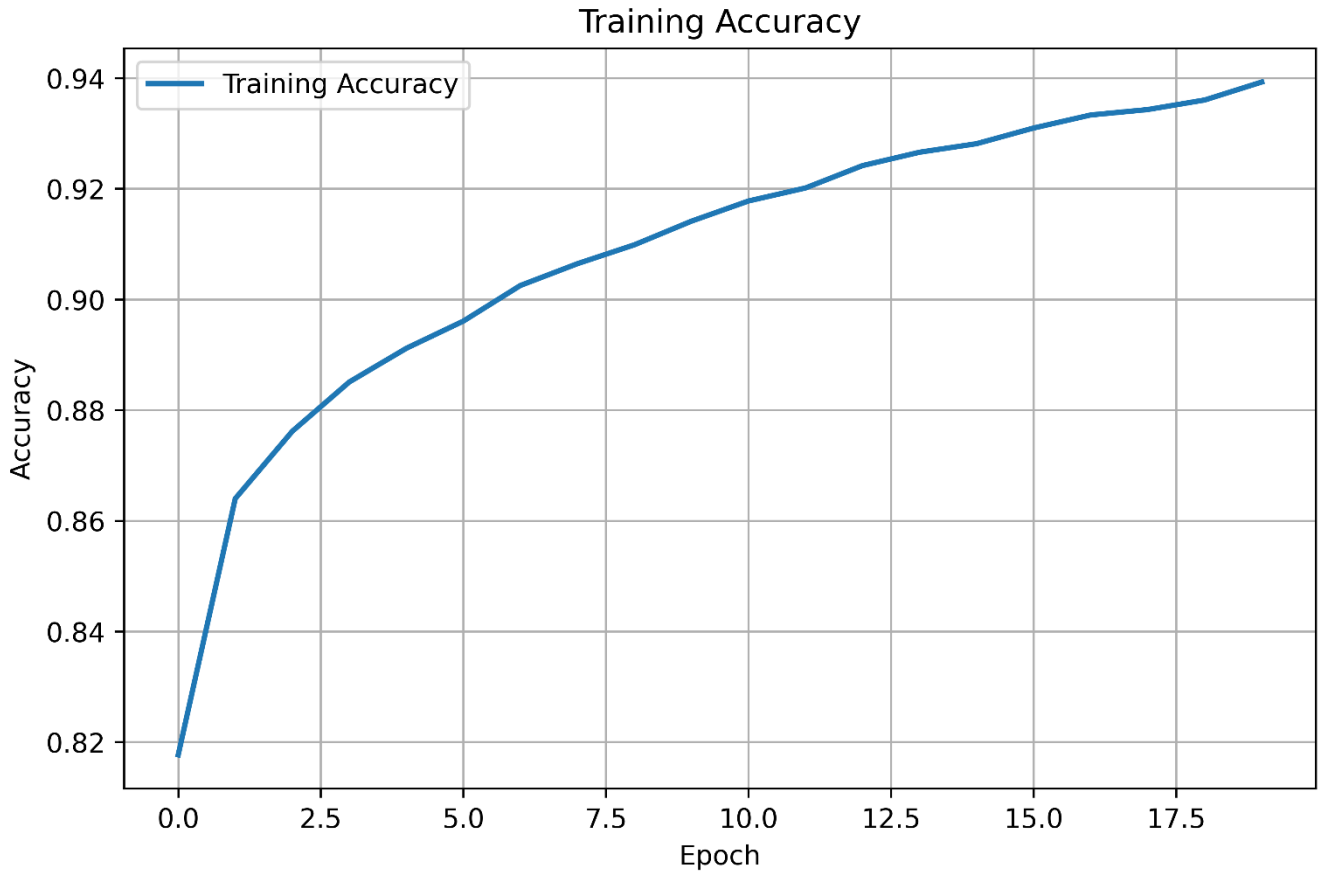
2. Class Distribution



Inference:

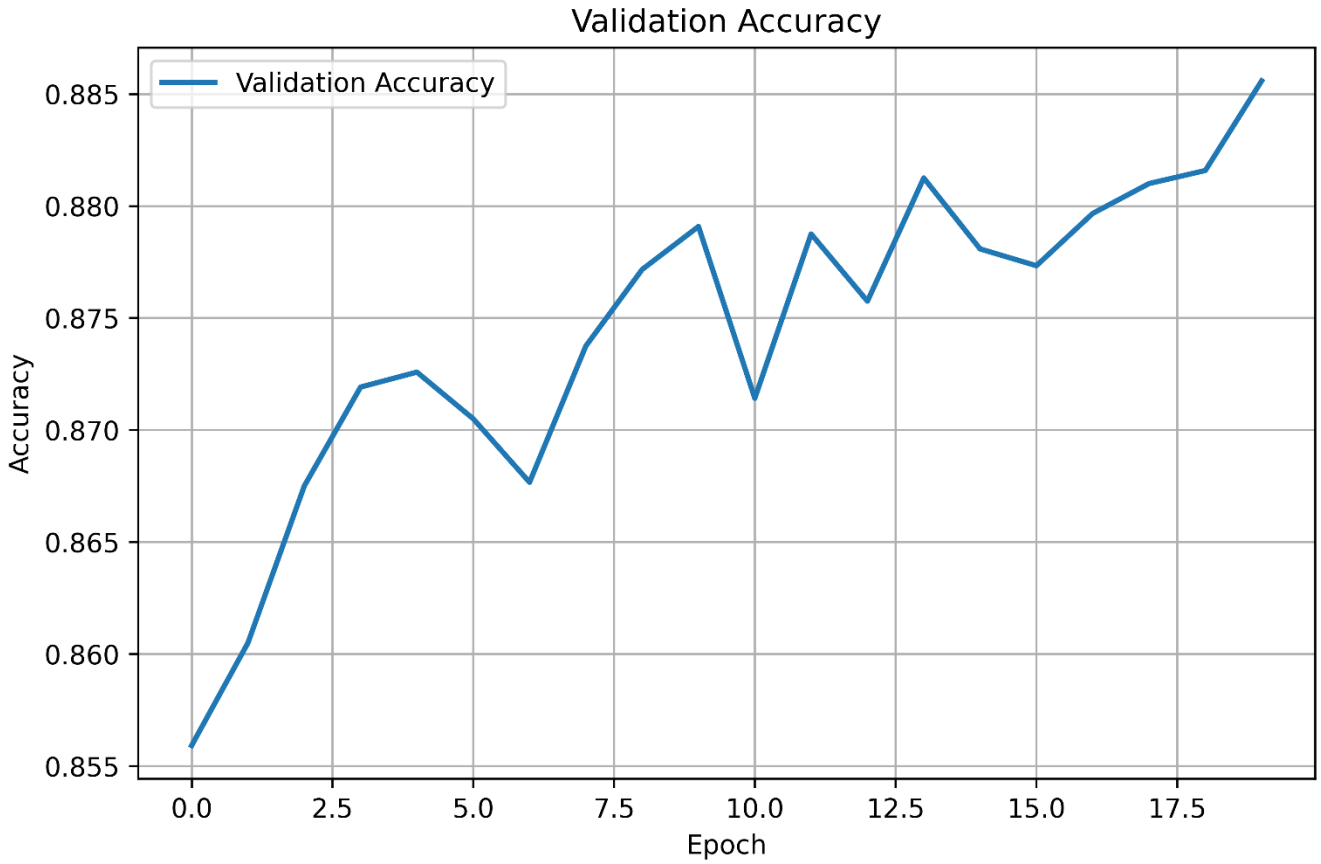
The class distribution is almost equal for all the categories in the dataset. Since every class has nearly the same number of images, the model is trained on a balanced dataset without any class imbalance

3. Training Accuracy vs Epoch



Interpretation:
The training accuracy increases continuously with each epoch and gradually reaches a stable value. This shows that the model is learning the patterns in the dataset effectively during training.

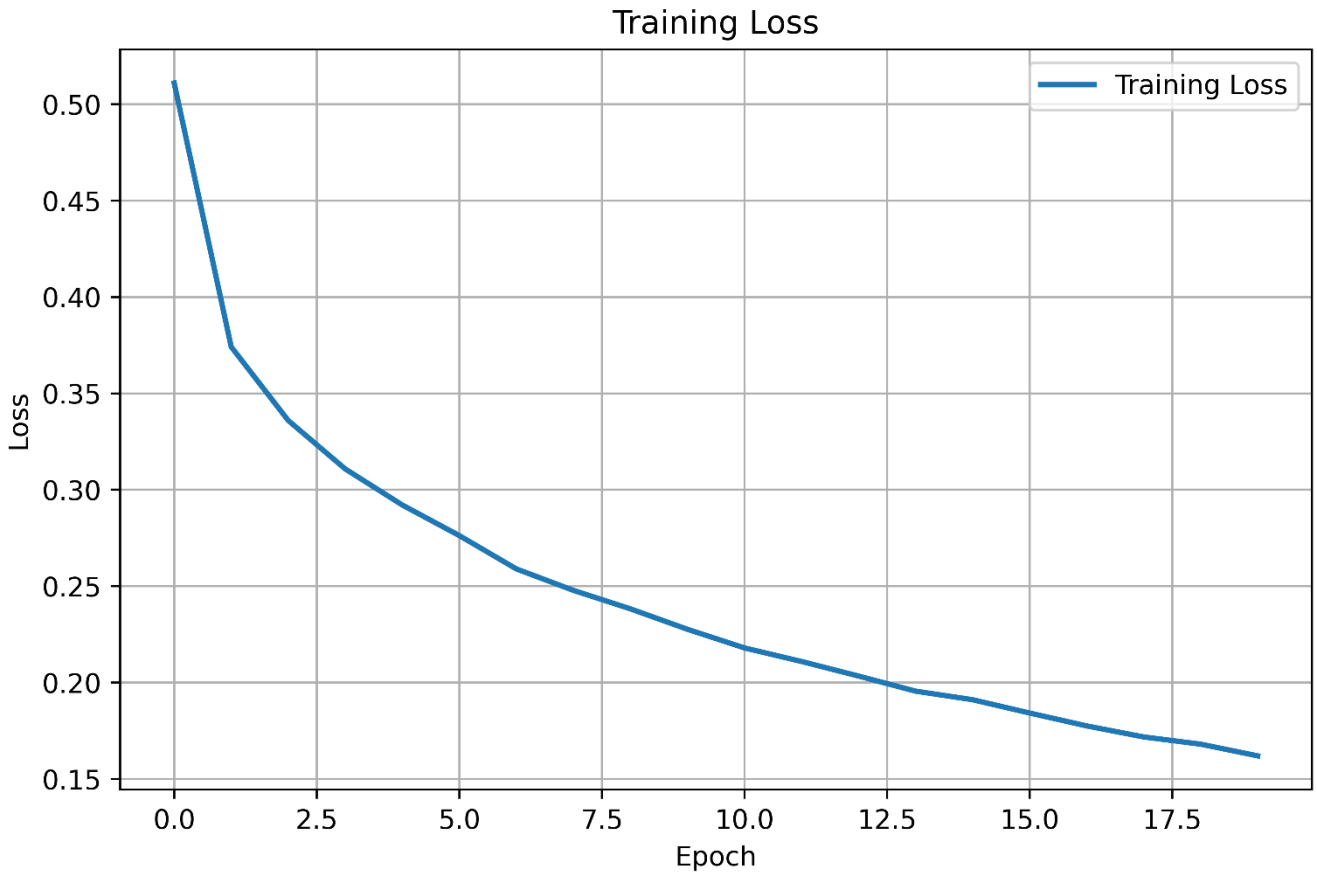
4. Validation Accuracy vs Epoch



Interpretation:

The validation accuracy increases gradually with small fluctuations and reaches about 88.5% in the final epoch. This indicates that the model performs well on unseen data and shows good generalization.

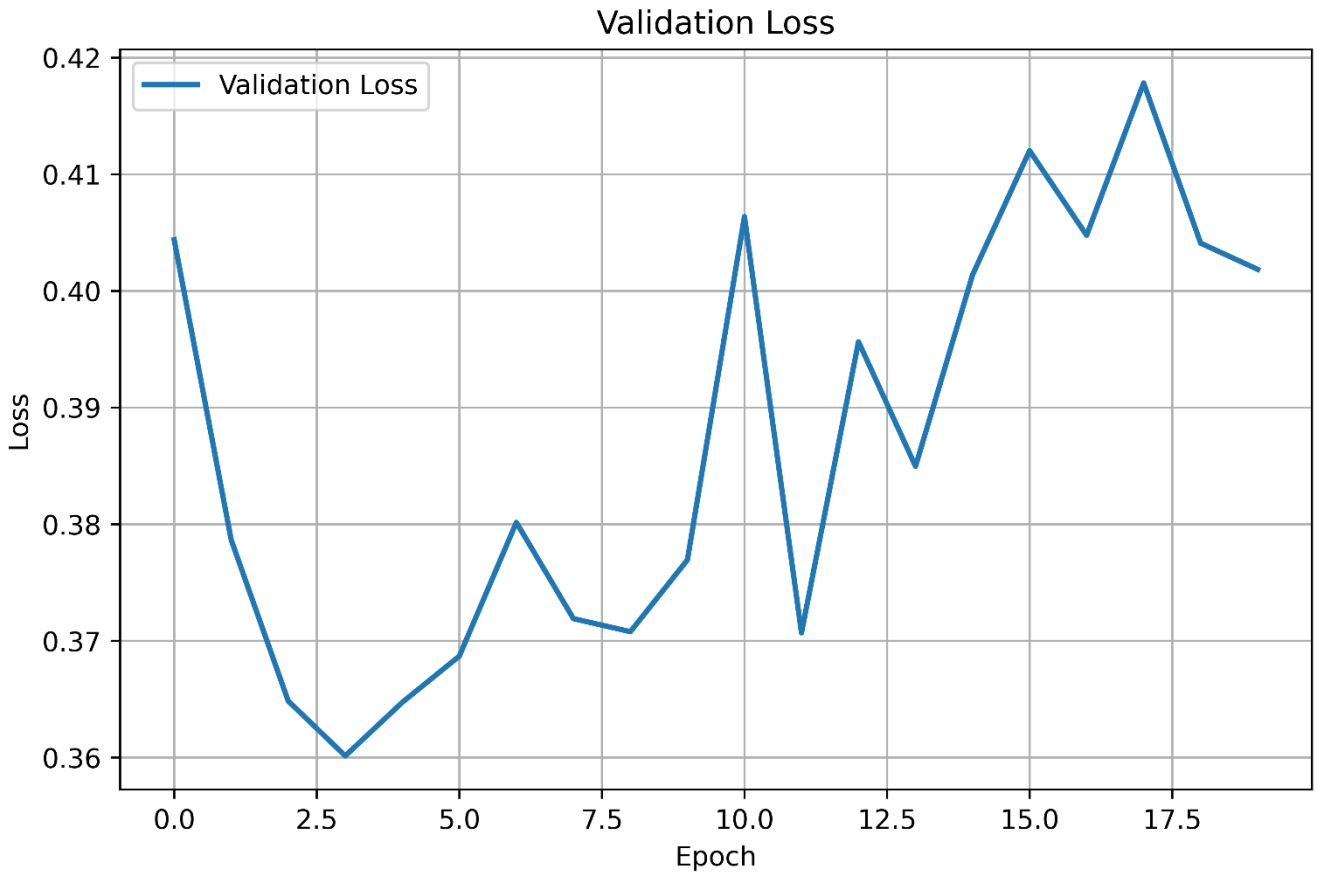
5. Training Loss vs Epoch



Interpretation:

The training loss decreases continuously as the number of epochs increases. This shows that the model is reducing prediction errors and learning the dataset effectively during training.

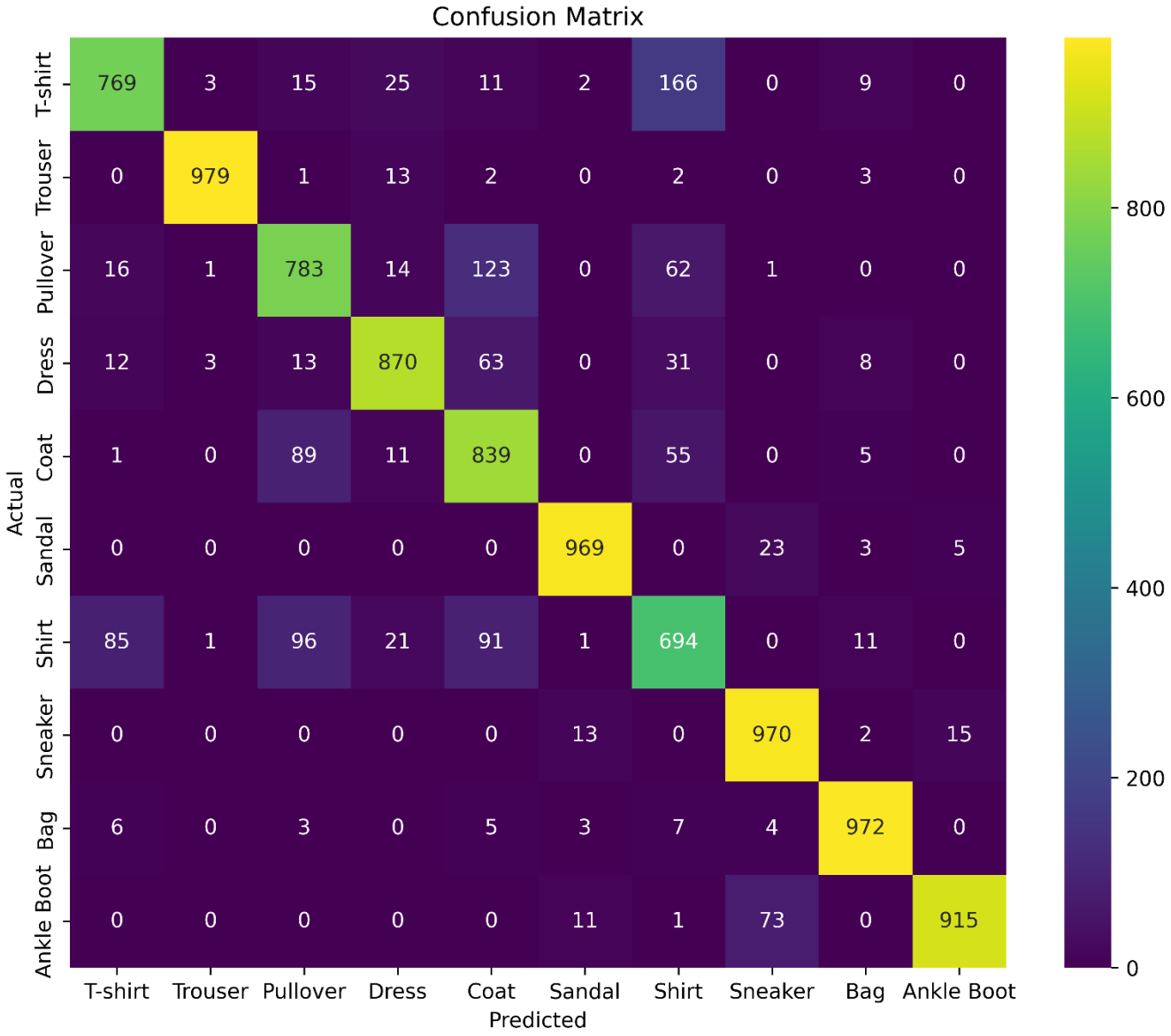
6. Validation Loss vs Epoch



Interpretation:

The validation loss decreases during the initial epochs and then shows small fluctuations in the later epochs. This indicates that the model has learned the important features, although slight variations are observed while validating on unseen data.

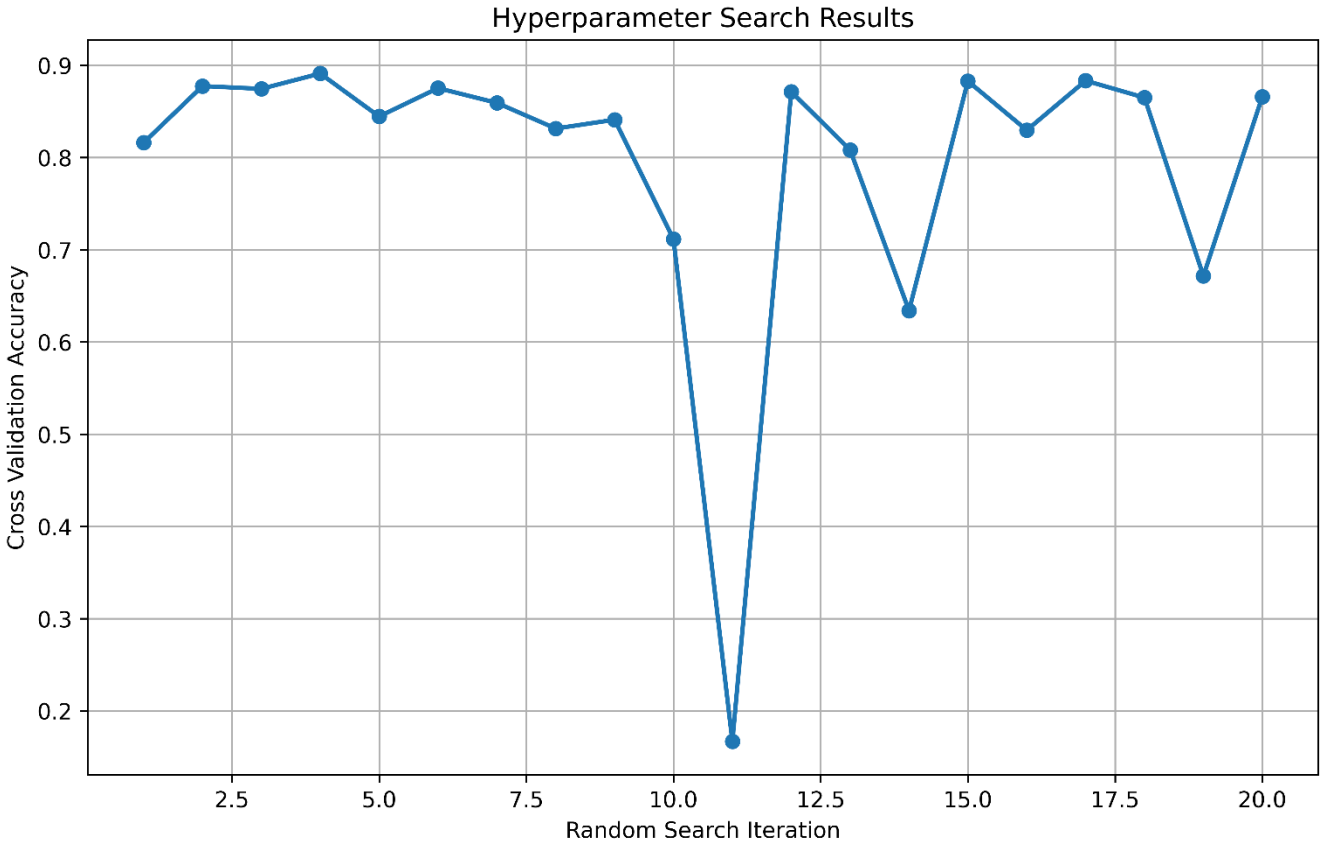
7. Confusion Matrix



Interpretation:

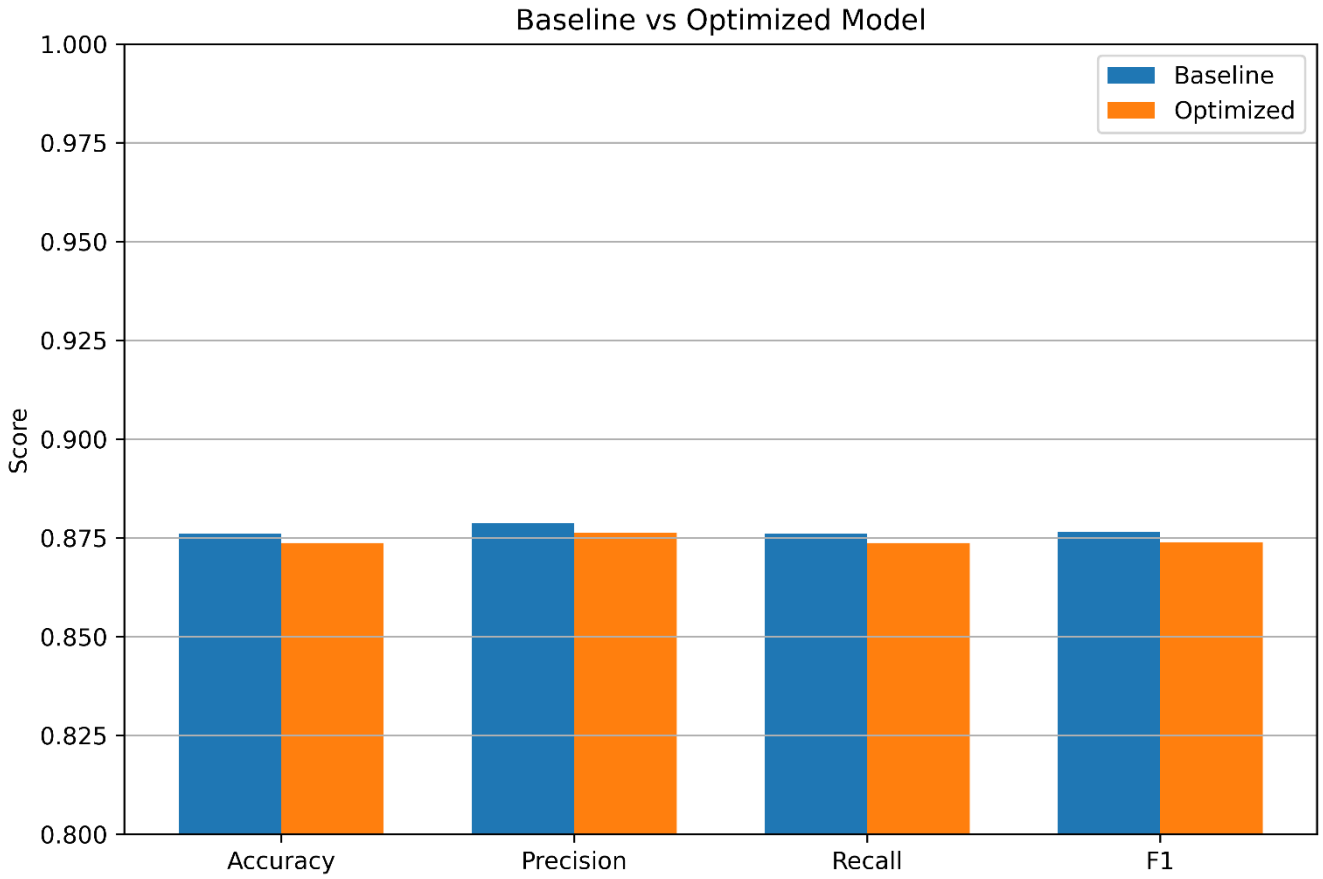
Most of the predictions are concentrated along the diagonal of the confusion matrix, which indicates that the model correctly classified most of the images. Some misclassifications are observed between similar clothing categories such as T-shirt, Shirt, Pullover, and Coat, because these classes have similar visual features.

8. Hyperparameter Search Results



Interpretation:
The cross-validation accuracy varies for different hyperparameter combinations, showing that the selected hyperparameters significantly affect the model performance. The highest cross-validation accuracy is achieved for one of the random search iterations, which is selected as the best configuration

9. Best Model Accuracy Comparison



Interpretation:
The baseline and optimized models achieved very similar performance for Accuracy, Precision, Recall, and F1-score. The baseline model performed slightly better than the optimized model, indicating that hyperparameter optimization did not improve the testing performance in this experiment.

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.1
Batch Size	32
Optimizer	SGD
Activation Function	Tanh
Epochs	30
Dropout	0.0
Cross-validation Accuracy	0.8911
Testing Accuracy	0.8737

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8760	0.8737
Precision	0.8787	0.8763
Recall	0.876000	0.873700
F1-score	0.876489	0.873928
Training Time	89.055140	131.150079

11. Discussion

Answer the following:

1. **Which optimization method was used?**

In this experiment**,** Randomized Search Cross-Validation (RandomizedSearchCV) was used for hyperparameter optimization. It randomly selected different combinations of hyperparameters and evaluated each combination using 5-fold cross-validation. Finally, it selected the combination that gave the best cross-validation accuracy.

2. **Which hyperparameters were selected?**

The best hyperparameters selected during the optimization process are:

- Hidden Layers : 1
- Hidden Neurons : 128
- Learning Rate : 0.1
- Batch Size : 32
- Optimizer : SGD
- Activation Function : Tanh
- Epochs : 30
- Dropout : 0.0

These hyperparameters achieved a cross-validation accuracy of 89.11%.

3. **Did optimization improve performance?**

No. The optimized model did not improve the testing performance. The baseline model achieved a testing accuracy of 87.60%, whereas the optimized model achieved 87.37%. Although the optimized model obtained a higher cross-validation accuracy (89.11%), it did not perform better on the test dataset.

4. **Which hyperparameter had the greatest impact?**

Among all the hyperparameters, the learning rate had the greatest impact on the model performance because it controls how the model learns during training. The optimizer and activation function also influenced the final accuracy by affecting the learning process and convergence of the model.

5. **Compare Grid Search and Randomized Search.**

Grid Search tries all possible combinations of the given hyperparameters to find the best model. Since it checks every combination, it takes more time and requires more computation.

Randomized Search tries only a random set of hyperparameter combinations instead of checking all of them. Because of this, it is much faster and is suitable for large search spaces. In this experiment, Randomized Search was used because it reduced the training time while still finding a good set of hyperparameters.

6. **Recommend the best MLP configuration.**

Based on the experimental results, the recommended MLP configuration is:

- Hidden Layers : 1

- Hidden Neurons : 128
- Learning Rate : 0.1
- Batch Size : 32
- Optimizer : SGD
- Activation Function : Tanh
- Epochs : 30
- Dropout : 0.0

Although this configuration achieved the highest cross-validation accuracy (89.11%), the baseline model obtained a slightly better testing accuracy (87.60%). Therefore, for this dataset, the baseline model performed slightly better on unseen test data.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.